

Methodology

The methodology of this project is based on the Waterfall Model, a traditional and linear software development approach where each phase must be completed before moving to the next.

This model is ideal for academic-level web development projects where the requirements are clearly defined from the start. Since our phishing detection web app has specific objectives and features, the Waterfall Model ensures structured planning, implementation, and delivery.

→ Phase 1: Requirement Gathering

In the first phase, we'll collect both functional and non-functional requirements for the system. This included understanding how phishing links behave, which rule-based patterns to detect, how users would interact with the web interface, and how the chatbot would assist them.

We will also finalize the technologies to be used, like HTML/CSS and JavaScript for the frontend and Python (Flask) for the backend processing.

Additionally, we will explore the feasibility of integrating a Machine Learning model for more intelligent phishing detection based on real-world URL datasets (e.g., from Kaggle or PhishTank).

→ Phase 2: System Design

This stage focused on designing the system's architecture. It involved planning the user interface for link input, backend APIs for analyzing links using rule-based logic, and integration with blacklist APIs (e.g., Google Safe Browsing or PhishTank).

A chatbot module will also be designed for user interaction. In parallel, we will identify and define the features to be extracted for ML, such as URL length, use of special characters, presence of IP addresses, and keyword patterns.

A plan will be made to connect the ML model (**Random Forest Classifier**) to the backend process.

→ Phase 3: Implementation

We will build the frontend using HTML, CSS, and JavaScript to create a user-friendly web interface. The backend will be developed using Python and Flask, where phishing detection logic will be implemented using predefined rules and blacklist checks.

Apart from that, the ML model predicts whether a link is phishing or not based on extracted features. After analysis, results will be returned to the frontend, along with chatbot responses.

→ Phase 4: Testing

We will test the system using different kinds of URLs i.e. safe, shortened, phishing, and obfuscated. However, we will also evaluate the accuracy of both the rule-based logic and the ML predictions, ensuring that response handling and the user interface worked as expected.

→ Phase 5: Deployment & Maintenance

After successful testing, the web app will be deployed on a local server and made accessible via a browser. In the future, we plan to continue updating detection rules, retraining the ML model with updated datasets, and upgrading the chatbot to an AI-powered system (e.g., Dialogflow or ChatGPT API).

Methodology Tangible & Tractable

The adopted methodology is tangible and tractable, as it follows a structured Waterfall Model with clearly defined and measurable phases.

- Each phase (requirements, design, implementation, testing) produces verifiable outputs such as documents, code modules, and test results.
- The phishing detection logic and ML integration are implemented incrementally, making progress easy to track.
- Tools and technologies used (Flask, HTML/CSS/JS, Scikit-learn) are well-supported and practical for student-level development.